▸ Aufgabe 1: Die Programmiersprache Goto

1. Schreiben Sie ein *while*-Programm, das als Eingabe eine Zahl n bekommt und dasselbe berechnet wie folgendes *goto*-Programm:

```
        z ← 0
loo:    if n ≤ 1
            goto ende
        end
        if n % 0
            n ← n / 2
        else
            n ← 3 · n + 1
        end
        ++z
        goto loo
ende:   return z
```

Anmerkung: Dieses Programm berechnet die Länge der sogenannten "Collatz-Folge". Ob das Programm für alle n ∈ $\mathbb{N}$ hält, ist eine bislang unbewiesene Vermutung.

Lösung:

```
z ← 0
while n > 1
    if n % 0
        n ← n / 2
    else
        n ← 3 · n + 1
    end
    ++z
end
return z
```

2. Schreiben Sie ein *goto*-Programm, dass als Eingabe eine Liste a bekommt und genau dasselbe berechnet wie das folgende *while* -Programm:

```
n ← count(a)
if n > 1
   for i ← 1, …, n
      k ← i
      j ← k+1
      while j ≤ n
         if a_j < a_k
            k ← j
         end
         ++j
      end
      x ← a_i
      a_i ← a_k
      a_k ← x
   end
end
return a
```

Anmerkung: Ihr Programm soll keine *loop*-, *while*- oder *for*-Schleifen enthalten.

Lösung:

```
   n ← count(a)
   if n > 1
      i ← 1
L1:   k ← i
      j ← k+1
L2:   if j ≤ n
         if a_j < a_k
            k ← j
         end
         ++j
         goto L2
      end
      x ← a_i
      a_i ← a_k
      a_k ← x
      ++i
      if i ≤ n
         goto L1
      end
   end
   return a
```

Zusatzfrage: Was genau macht dieses Programm?

3. Schreiben Sie ein *goto*-Programm, dass als Eingabe eine Zahl n bekommt, und genau dann hält, wenn n eine gerade Zahl ist; in diesem Fall soll das Programm die Zahl 42 zurückgeben (ansonsten soll es sich aufhängen 😜).

Anmerkung: Ihr Programm soll keine *loop*-, *while*- oder *for*-Schleifen enthalten.

```
      if n % 2 = 0
         return 42
      end
L:    goto L
```

▸ Aufgabe 2: Reterna verstehen

Was berechnen folgende Reterna-Funktionen?

1. `f1(n, m) = n ? f1(n−1, m) + m : 0`

$f1(n, m) = n \cdot m$

2. `f2(n, m) = m ? f2(n+1+1, m−1) : n`

$f2(n, m) = n + 2 \cdot m$

3. `f3(n) = n ? f3(n−1) + n : 0`

$$f3(n) = \sum_{i=0}^{n} i = \frac{n \cdot (n+1)}{2}$$

Beispiel:

$$f3(3) = \underbrace{f3(2)}_{f3(1)+2} + 3 = \underbrace{f3(1)}_{f3(0)+1} + 2 + 3 = \underbrace{f3(0)}_{0} + 1 + 2 + 3 = 6$$

$f3(4) = f3(3) + 4 = 10$

Im Folgenden dürfen Sie alle Rechenoperationen, logischen Operationen und Vergleiche verwenden, die im Video vorgestellt worden sind.

▸ Aufgabe 3: Reterna Programmieren

1. Schreiben Sie eine Reterna-Funktion **greater10**(n), die *true* zurückgibt, wenn n größer als 10 ist, und ansonsten *false* zurückgibt.

```
greater10(n) = n − 10 ? true : false
```

oder einfacher:

```
greater10(n) = n > 10
```

😜

2. Schreiben Sie eine Reterna-Funktion **even**(n), die *true* zurückgibt, wenn n gerade ist, und ansonsten *false* zurückgibt.

```
even(n) = n % 2 ? false : true
```

3. Schreiben Sie eine Reterna-Funktion **fac**(n), die die Fakultät n! = 1 · 2 · 3 · … · n zurückgibt. Beachten Sie dabei: 0! = 1

```
fac(n) = n ? fac(n−1) · n : 1
```

4. Schreiben Sie eine Reterna-Funktion **log2**(n), die berechnet, wie oft man n halbieren kann (mit Abrunden), bis man die Zahl 1 erreicht . Dabei sei n ≥ 1, und wir definieren log2(1) = 0.

Hinweis: der Ausdruck n/2 berechnet den Ganzzahl-Anteil bei der Division von n durch 2, d.h. n/2 rundet ab.

```
log2(n) = n > 1 ? log2(n/2) + 1 : 0
```

▸ Aufgabe 4: Reterna ⇒ Sprache

Sei Σ ein Alphabet das − eventuell neben weiteren Zeichen − auch den Buchstaben "A" enthält. Die folgenden Reterna-Programme erhalten als Eingabe einen als Zahl kodierten String a ∈ Σ*. Geben Sie zu jedem der folgenden Reterna-Programme an, welche Sprache das Programm akzeptiert, und zwar indem Sie entweder die Notation L = {…} oder − wenn das geht − einen regulären Ausdruck angeben:

1.

```
Language1(a) = len(a) ? a[0] = "A" : false
```

L = {w ∈ Σ* | w beginnt mit "A"} = AΣ*

2.

```
Language2(a) = L2(a, len(a)−1)
L2(a, i) = (a[i] = "A") and (i > 0 ? L2(a, i−1) : true)
```

L = {w ∈ Σ* | w ≠ ε und w enthält nur den Buchstaben A} = AA* = A^+

Zusatzfrage: Wie muss ich das Programm ändern, damit es die Sprache L = A* akzeptiert?
Antwort:

```
Language2(a) = len(a) ? L2(a, len(a)−1) : true
L2(a, i) = (a[i] = "A") and (i > 0 ? L2(a, i−1) : true)
```

3.

```
Language3(a) = L3(a, 0, len(a)−1)
L3(a, i, m) = i < m ? (a[i] ≤ a[i+1]) and L3(a, i+1, m) : true
```

L = {w ∈ Σ* | die Buchstaben in w sind aufsteigend nach ihrem code sortiert}